

Florence Web App Build Specification

Version 1.0 | 20 September 2026

1 Product decision

Build Florence as a responsive web app for an event planning team. It helps a planner choose coordinated colors, assemble flower combinations, apply the choices to an event concept, and prepare a client presentation. The main workspace should feel like a calm design studio, with large color swatches, useful flower imagery, and clear editing controls.

Use **Next.js App Router**, **React**, **TypeScript**, **Tailwind CSS**, **shadcn/ui**, **Supabase**, and **Culori**. Host on a Vercel plan suitable for commercial use. Use deterministic matching rules in the first release. A language model is not required to generate palettes or rank flowers.

This document is the implementation contract. It includes product scope, UI behavior, algorithms, data contracts, permissions, seed content, delivery stages, and acceptance criteria. A coding model should implement one milestone at a time and preserve the decisions here. The Markdown edition is the canonical input for a coding model; the Word edition contains the same specification for review.

Operating assumptions

- Florence is the working product name.
- The first release serves one event planning company. Workspace isolation is implemented even though the initial UI supports one workspace per user.
- Staff use the app on laptops, tablets, and phones. Clients receive an exported concept; they do not need accounts in the first release.
- Launch language is English. The workspace selects its currency and time zone during setup; do not infer location, currency, or flower season from the browser.
- Flower imagery, color samples, and design tags are curated content. Price, stock, local seasonality, and event suitability require separate evidence.
- The first release is a design decision tool. Procurement, exact stem calculations, invoicing, and event logistics are outside its scope.

Definition of the main outcome

A planner can create an event, choose five colors, understand their relationship, select a combination of flowers, assign colors to event elements, save the work, and print a readable concept sheet. The full flow must work at 390 px and 1440 px viewport widths.

2 Users and success criteria

The planner creates and edits concepts. A senior designer reviews combinations and records practical notes. A workspace viewer can inspect projects and print concepts but cannot change them. An administrator has editor permissions and manages workspace configuration through the deployment setup process in version one.

Pilot targets are product goals, not measured results:

- A first-time planner completes one concept in ten minutes after a short introduction.
- A returning planner can change a primary color and inspect updated flower suggestions in under one minute.
- Staff can tell why a flower was suggested without reading a numeric formula.
- Refreshing a successfully saved project preserves all choices.
- A user outside a workspace cannot read or mutate its projects, including through direct API requests.

- Five event planners complete the main flow during a pilot. Record confusion, time, and missing content before expanding features.

3 Release scope

P0 required for the first release

1. Staff authentication, workspace membership, and editor or viewer permissions.
2. Project list with search, status filters, create, duplicate, archive, and restore.
3. Event brief with event type, date, location, environment, style, client display name, and design notes.
4. Five-color palette editor with HEX entry, color picker, harmony modes, locked colors, proportions, named presets, and undo.
5. Flower catalog with color variants, search, filters, flower details, and ranked suggestions.
6. A combination builder with flower roles, visual rationale, and separate practical review notes.
7. A concept board with color assignments, selected flowers, public presentation notes, and a saved status.
8. Private project persistence, conflict detection, and explicit retry after save errors.
9. A print layout using the browser print dialog, including its Save as PDF option where supported.
10. A demo mode using supplied fixtures when backend credentials are absent in local development.

P1 after the main flow is validated

Add photo color extraction, private inspiration uploads, reusable palette libraries, comparison of two concepts, supplier quotes, regional availability records, workspace invitations, and client share links. Each requires its own permissions and acceptance criteria before implementation.

Outside the first two releases

Do not add a marketplace, payment processing, generative event renderings, flower identification from photos, real-time collaborative editing, native mobile apps, 3D room planning, inventory purchasing, or automatic promises about allergy safety or flower longevity. No placeholder buttons for these features should appear in the P0 interface.

4 Main user journey

1. The planner selects **New event** from Projects.
2. They enter an event name and optionally an event type, date, location, environment, and style. Only the event name is required to create a draft.
3. Florence opens the Palette tab with the Garden Romance preset. The planner may replace it, enter a primary HEX color, or select a different harmony mode.
4. They lock any chosen swatch and select **Generate alternatives**. Locked colors remain identical.
5. The Flowers tab ranks catalog variants against the palette. The planner sees a color match explanation, design role, and practical status for each candidate.
6. They add a focal flower, a supporting flower, an optional filler or line flower, and optional foliage. The combination review updates without requiring a page reload.
7. On Concept, they assign colors to backdrop, linen, floral emphasis, stationery, and accents. They add client-facing notes.

8. They wait for **Saved**, open **Preview concept**, and select **Print or save PDF**.

The app must also support starting from a favorite flower. From a flower detail, **Use as primary color** opens a project chooser. After choosing a project, a confirmation preview shows the palette change. Applying it sets the primary swatch and generates the other unlocked slots; it never silently replaces locked colors or selected flowers. If primary is locked, require the planner to unlock it in this preview before Apply becomes available.

5 Navigation and screens

Route	Purpose	Main action
/login	Staff sign-in	Send sign-in link
/auth/callback	Complete Supabase authentication	Redirect to Projects
/projects	Event project list	New event
/projects/new	Create an event brief	Create event
/projects/[id]?tab=palette	Edit colors	Generate alternatives
/projects/[id]?tab=flowers	Find and combine flowers	Add to combination
/projects/[id]?tab=concept	Compose event concept	Preview concept
/projects/[id]?tab=brief	Edit event information	Save brief
/projects/[id]/print	Private print preview	Print or save PDF
/flowers	Browse catalog without project context	View flower

The root route redirects to Projects when signed in and Login otherwise. Unknown routes display a useful not-found page. An inaccessible project returns the same not-found treatment as a nonexistent project.

Project list

Show a title, a short subtitle, search, status filter, and New event button. Each card displays event name, date or Date not set, style, five mini swatches, status, and last updated time. Use a two-column grid on tablet and a three-column grid on wider desktops when card width remains at least 280 px. Phone cards stack vertically.

Search matches event name and client display name, case-insensitively. Default sorting is updated time descending. Archived projects are hidden unless the Archived filter is selected. Duplicate creates a new draft called the original name plus Copy, with new IDs, copied design choices, and no copied practical confirmations.

Empty state: “Create your first event concept.” Include one New event action and one View demo action only in demo mode. Loading uses card skeletons. An error shows Retry while preserving the current filters.

Event editor shell

Use a persistent project title, a back link, status, save indicator, and Preview concept action. Below this, show tabs for Palette, Flowers, Concept, and Brief. Store the tab in the URL. Browser back and refresh must preserve the selected tab.

Desktop navigation uses a 216 px sidebar with Florence, Projects, and Flower library. On screens below 1024 px, replace the sidebar with a compact top bar and a labeled menu button. Keep project tabs visible and horizontally scrollable if needed; scrolling must not hide the focused tab.

6 Visual design system

The visual direction is a modern floral design studio. Use warm white backgrounds, deep green controls, generous spacing, and restrained serif headings. The event palette is the main source of bright color. Keep the application chrome stable when a user edits an event palette.

Token	Value	Use
canvas	#FAF8F5	Page background
surface	#FFFFFF	Cards and dialogs
ink	#242B27	Main text
muted	#626C64	Secondary text
brand	#264C3B	Primary buttons and active controls
brandSoft	#EDF3EE	Selected backgrounds
border	#DDDCD5	Decorative dividers
controlBorder	#7A847C	Input boundaries when needed
danger	#A52A37	Error text and destructive actions
warning	#775B1A	Practical review notices

Use Inter for body text and controls, and locally hosted Cormorant Garamond for the Florence wordmark and main page headings. Include font licenses. Fall back to system sans-serif and Georgia if assets fail. Body text is 16 px with 1.5 line height. Desktop page headings are 36 px; mobile headings are 28 px. Section headings are 22 px. Supporting labels may be 14 px; essential text must not be smaller.

Use an 8 px spacing rhythm with 4 px for compact gaps. Cards have 16 px radius, inputs 10 px radius, and buttons 10 px radius. Default card padding is 24 px desktop and 16 px mobile. Use subtle shadows only for floating sheets and dialogs. Use Lucide icons with visible labels for primary actions. Avoid decorative gradients, glass effects, large hero sections, and animated backgrounds.

Flower images use a consistent 4:3 crop. Do not recolor flower photographs with CSS to fabricate variants. Missing images display a neutral botanical placeholder, the flower name, and the variant swatch.

Responsive layout rules

Width	Palette workspace	Flower workspace	Navigation
320 to 767 px	Stacked swatch rows and full-width controls	One-column cards and combination sheet	Top bar and tabs
768 to 1023 px	Five-swatch strip and stacked inspector	Two-column cards	Top bar and tabs
1024 to 1279 px	Main area with inspector below	Two-column cards and combination drawer	Sidebar
1280 px and above	Main area plus 320 px inspector	Card grid plus 320 px combination panel	Sidebar

Main content has a maximum width of 1600 px. Use 16 px outer padding on phones, 24 px on tablets, and 32 px on desktops. Do not squeeze three panels into a narrow laptop screen. Drawer state must not reset the selected combination.

Mobile editing uses full-width controls and at least 44 by 44 px touch targets. A sticky bottom action bar must account for safe-area insets and reserve matching content padding. At 200 percent zoom, layouts reflow instead of hiding controls.

Desktop palette layout

Florence Projects Flowers	Event name		Saved	Preview concept	
	Palette Flowers Concept Brief				
	Primary		Support	Accent	Neutral
	Foliage				
	[large color swatches with labels and HEX]				
	[proportion strip totaling 100 percent]				
	Harmony mode		Selected swatch		
	Generate alternatives		Picker and HEX input		
	Undo		Lock and role label		
Preset gallery		Explanation			

Phone palette layout

Menu	Event name	Preview	
Palette	Flowers	Concept	Brief
Primary	[swatch]	HEX	Lock
Support	[swatch]	HEX	Lock
Accent	[swatch]	HEX	Lock
Neutral	[swatch]	HEX	Lock
Foliage	[swatch]	HEX	Lock
Harmony mode			
Generate alternatives			Undo
Proportions and explanation			

7 Palette editor behavior

Every project has exactly five swatches with stable IDs and fixed roles: primary, support, accent, neutral, and foliage. Their suggested visual proportions are 40, 25, 15, 15, and 5 percent. These describe the concept’s visual emphasis, not stem counts or procurement quantities.

Each swatch has a user-editable display name, normalized six-digit HEX value, lock state, and integer proportion. Clicking a swatch selects it. Clicking Copy HEX copies only its HEX value and announces success. The color picker and HEX field update one shared draft state.

Accept three- or six-digit HEX with an optional leading hash, normalize to uppercase six-digit HEX, and reject alpha values, CSS expressions, and invalid strings. Do not replace a valid saved value while the input contains a partial invalid edit. Display “Enter a HEX color such as #D8A7B1.”

Changing HEX manually is allowed even when locked; the lock only protects against generation and preset application. Manual primary edits reset the generation counter to zero but do not immediately change other swatches. The Generate alternatives action performs that change explicitly.

Preset selection opens a preview. Applying a preset replaces only unlocked swatches, keeps existing proportions, and resets the generation counter. Show “Two locked colors will stay unchanged” when relevant. Disable generation and preset application when all swatches are locked. Palette changes never automatically remove chosen flowers.

Keep the last 20 palette states in memory for Undo during the editing session. One generation, preset application, or committed picker edit is one history entry. Undo does not restore unrelated brief or flower changes. Refresh starts a new undo history.

Proportions must each be between 0 and 100 and sum to 100 before the palette edit is committed. Show the current total. Offer **Normalize to 100%** using largest-remainder rounding: scale all nonnegative values to 100, floor each, and allocate the remaining percentage points by descending fractional remainder, ties in role order. For five zero values, restore the default proportions.

8 Color generation and matching rules

These are Florence design heuristics. They generate starting points for a designer; they do not prove that a combination is universally attractive. Keep constants in one versioned module and cover them with deterministic tests.

Canonical color handling

Store normalized sRGB HEX as the source of truth. Convert through Culori to OKLCH for generation and OKLab for distances. Culori provides color conversion, color differences, and gamut mapping; use its supported API rather than writing conversion matrices from memory. [Culori API](#)

After generation, map into sRGB gamut and serialize to HEX. Recompute derived values from that resulting HEX. Never alter a locked HEX while mapping other colors. The palette's neutral and foliage slots are styling anchors; they do not have to follow the selected harmony mode.

Deterministic generation contract

Input: current five swatches, harmony mode, and integer generationIndex. Output: the five updated swatches, warnings, and next generationIndex. Same input must produce the same output. No Math.random calls.

The primary HEX is always the generation anchor and remains unchanged, even when its lock is off. Its unlocked state still permits replacement by a preset. Generation modifies only unlocked support, accent, neutral, and foliage slots.

Convert the primary to OKLCH. Let h be its hue, L its lightness, and C its chroma. If hue is undefined or C is below 0.02, use h = 30 degrees for generated colored slots and state "A warm accent direction was used for this neutral base." Clamp generated colors only; preserve the primary itself.

Define k = generationIndex modulo 3. Use lightness shift [0, 0.05, -0.05][k] and chroma multiplier [1, 0.85, 1.1][k]. The first click uses k = 0, then increments the counter. Define c = clamp(max(C, 0.08) times multiplier, 0.05, 0.18). Wrap every hue into the interval [0, 360).

Mode	Support hue	Accent hue	Support lightness	Accent lightness
analogous	h minus 30	h plus 30	clamp(L plus 0.12 plus shift, 0.35, 0.90)	clamp(L minus 0.12 plus shift, 0.30, 0.85)
complementary	h	h plus 180	clamp(L plus 0.18 plus shift, 0.35, 0.90)	clamp(L minus 0.05 plus shift, 0.30, 0.85)
splitComplementary	h plus 150	h plus 210	clamp(L plus 0.10 plus shift, 0.35, 0.90)	clamp(L minus 0.08 plus shift, 0.30, 0.85)
triadic	h plus 120	h plus 240	clamp(L plus 0.10 plus shift, 0.35, 0.90)	clamp(L minus 0.08 plus shift, 0.30, 0.85)
monochromatic	h	h	clamp(L plus 0.20 plus shift, 0.35, 0.92)	clamp(L minus 0.20 plus shift, 0.25, 0.85)

Use support chroma 0.7 times c and accent chroma c. For monochromatic mode only, use the actual primary C clamped to [0, 0.18] instead of c; a gray seed therefore stays gray. Neutral is OKLCH(0.95, 0.012, h). Foliage is OKLCH(0.52 plus shift, 0.07, 145).

If the OKLab distance between two resulting swatches is below 0.025, show "Some shades are very similar." Keep the values; do not loop forever trying to find distinct shades. If locks prevent the requested relationship, explain that the mode applies only to regenerated colors.

For a complementary or triadic relationship, the label describes the configured OKLCH hue rotation. Do not imply that every artist's color wheel will return the same HEX complement.

Accessibility contrast

Color harmony and readable text are separate calculations. On a swatch, choose black or white label text by whichever has the higher WCAG contrast ratio. For concept stationery previews, flag normal text below 4.5:1 and large text below 3:1. Never call an entire palette accessible based on one pair. The application controls themselves target WCAG 2.2 AA, including keyboard operation and visible focus. [WCAG quick reference](#)

9 Flower catalog and recommendation model

A flower species may have several color variants. Ranking must operate on a variant, such as blush rose, rather than assuming all roses share one color. Foliage is a catalog item with a foliage role and is presented as foliage in the UI.

Catalog fields include stable ID, species key, common name, variant name, approximate HEX, image path, image credit, permitted design roles, texture tag, style tags, editorial review status, and optional practical facts. Practical facts include value, source, reviewed date, and regional applicability. Null means unknown.

Design roles are focal, support, filler, line, and foliage. A catalog item may permit multiple roles, but a selected item has one chosen role. These describe use in a design and are not immutable botanical classifications.

A catalog detail panel contains the image, variant name, swatch, allowed roles, stylistic rationale, source information, practical notes, and Add to combination. Already selected variants display Added. Clicking the image opens details; it never adds an item unexpectedly.

Filtering

Provide search by common name or variant, role, color family, and style. Use configured family tags rather than trying to parse color names. Show active filters and Clear all. Inside a project, add **Hide confirmed unavailable**. Unknown availability remains visible and labeled.

Do not implement a universal seasonal calendar. Location and event date alone cannot prove local supply. In P0, availability is an event-specific manual status: unknown, confirmed, or unavailable. Confirmation requires a source note and applies to that project date and location. Changing either invalidates confirmations back to unknown while preserving the old note as history text.

Candidate score

The following weights are product choices, not validated floristry measurements. Label the result **Visual fit**, never compatibility certainty or probability. Do not show a percentage sign.

1. Remove inactive catalog entries and apply the user's explicit filters. Exclude already selected variant IDs from the Add suggestions list.
2. For a foliage candidate, compare its HEX to the foliage swatch only. For any other candidate, compare to primary, support, accent, and neutral swatches with a proportion above zero. If none qualify, compare to primary.
3. Calculate d as the smallest Euclidean distance in OKLab to an eligible swatch. $\text{ColorFit} = \max(0, 1 - d \text{ divided by } 0.25)$.
4. If the user chose a target role, filter to candidates permitting it and set $\text{RoleFit} = 1$. Otherwise, $\text{RoleFit} = 1$ if the candidate fills a currently missing role, and 0.4 otherwise. Treat all five roles as missing when the combination is empty.

5. StyleFit = 1 if a candidate tag matches the selected event style, 0.5 if no event style is set or candidate tags are empty, and 0 otherwise.
6. TextureFit = 1 if its known texture is absent from the current combination, 0.4 if already present, and 0.5 if unknown.
7. Score = round(100 times (0.60 times ColorFit plus 0.20 times RoleFit plus 0.10 times StyleFit plus 0.10 times TextureFit)). Clamp to 0 through 100.
8. Sort by score descending, then common name, variant name, and stable ID ascending. Use a fixed English comparison in both browser and server tests.

Show High visual fit for scores 80 to 100, Good visual fit for 60 to 79, and Explore contrast for 0 to 59. The detail view may expose the integer and component values. Show up to two reasons derived from actual features: "Closest to your blush accent," "Adds a missing filler role," or "Adds an airy texture." Do not generate unsupported prose about scent, hardness, price, or availability.

If no result passes the filters, explain which filters are active and offer Clear filters. Do not invent catalog entries. Keep existing selections intact.

Combination review

Allow zero to eight distinct variants while drafting; ready concepts require at least one. Show thumbnails, chosen role, nearest palette swatch, remove control, and operational review status. A role selector lists only that variant's allowed roles. There is no drag-only interaction requirement.

Display advisory messages rather than one overall flower-pair score:

- No focal selected: "Choose a focal flower if you want a clear center of attention."
- More than two selected focal variants: "Several focal flowers may compete; review their proportions."
- No filler or line role: "Consider a filler or line flower for variation."
- No foliage: offer foliage suggestions without calling the arrangement incomplete.
- Every known texture is the same: "A different texture could add variation."
- Any unavailable item: "This concept includes a flower marked unavailable."

Balance, proportion, focal emphasis, and varied shapes and textures are useful floral design considerations. Florence uses these as prompts for a human designer, not physical compatibility guarantees. [University of Minnesota Extension floral design guidance](#)

Practical review separate from visual fit

For each selected flower, allow notes for availability, scent preference, outdoor exposure, conditioning, and supplier discussion. Default all practical checks to Needs review. A planner may mark a check Reviewed and enter a note; that means staff reviewed it, not that Florence verified the claim.

Never increase a visual fit score because stock or price is unknown. Do not produce claims about vase life, heat tolerance, fragrance, toxicity, or shared-water conditioning unless the catalog contains a reviewed source for the specific claim. Concept exports show unresolved practical notes in a short "To confirm" section.

10 Concept board and print output

Use a structured board, not a freeform canvas. The top area shows the event title and optional date, location, and style. Below it, display the palette with names, HEX values, and proportions, followed by a flower grid and element assignments.

Element assignments are backdrop, linen, floral emphasis, stationery, and accent details. Each references a swatch ID and has an optional material note up to 200 characters. Default assignments are primary, neutral, support,

neutral, and accent respectively. A small flat illustration of a table setting may use these fills; label it “Color placement preview.” It is not a photorealistic event rendering.

Maintain separate internalNotes and presentationNotes. Internal notes never appear in the print route. Presentation notes have a 2000-character limit. Do not include staff emails, save errors, internal record IDs, or numeric fit scores in the client concept.

The print route renders only the latest saved project. If edits are pending, Preview concept first flushes and confirms the save. On failure, keep the planner in the editor with Retry. A viewer may print without saving.

Print styling uses A4 portrait by default with 12 mm margins, readable 11 pt text, repeatable flower card sizing, and sensible page breaks. It must also print legibly to US Letter. Target two pages for a typical five-color, four-flower concept; allow more for eight flowers and longer notes. Use real text rather than a screenshot of the UI. Avoid splitting a flower card or heading from its first paragraph.

Print headings are Event concept, Color palette, Flower selection, Event elements, and To confirm when needed. Include the saved revision and print date. Footer note: “Colors are approximate on screen and in print. Confirm physical samples and flower availability before ordering.”

Use window.print from a client button. Label it Print or save PDF; do not promise an automatic PDF download. Ensure HEX labels and borders remain readable when the user’s printer omits background colors. Do not build a server PDF renderer in P0.

11 Recommended technical stack

Area	Decision	Reason
Web framework	Next.js App Router with React and TypeScript	One codebase for UI, routing, authenticated server access, and deployment
Styling	Tailwind CSS	Explicit responsive layouts and shared tokens
Components	shadcn/ui, one consistent primitive family	Editable component source for dialogs, tabs, sheets, forms, and menus
Data and authentication	Supabase Postgres and Auth	Relational data, staff sign-in, migrations, and row-level access control
Color calculations	Culori	Supported color-space conversion, distance functions, and gamut mapping
Validation	Zod	Shared request and project-payload validation
Editor state	React useReducer and context scoped to one editor	Predictable undo and draft changes without another global store
Icons	Lucide React	Consistent small UI icon set
Tests	Vitest and Testing Library; Playwright for flows	Pure-rule tests, component behavior, and browser coverage
Hosting	Vercel commercial-use plan	Simple Next.js deployment with preview environments
Print	Browser print CSS	Readable concept export without extra rendering infrastructure

Next.js supports the App Router with a TypeScript and Tailwind setup. Use a supported Node LTS compatible with the installed framework and hosting runtime. Resolve current stable dependencies once during project setup, record exact installed versions, commit the lockfile, and use frozen-lockfile installs in CI. Do not combine example code from different framework majors. [Next.js installation](#)

Tailwind's responsive utilities support the mobile-first layouts specified here. shadcn/ui distributes component code that can be adapted to Florence's tokens; the application still needs its own accessibility tests. [Tailwind responsive design](#) and [shadcn/ui documentation](#)

Supabase has a documented Next.js integration. Follow its current cookie-based server authentication pattern using @supabase/ssr, rather than inventing session handling. [Supabase Next.js guide](#)

Vercel's Hobby plan is restricted to personal non-commercial use. Do not budget this company tool as a free Hobby deployment. Check current Vercel and Supabase prices, quotas, backup options, and regions at implementation time; this specification commits to no recurring price. [Vercel Hobby plan rules](#)

Why this stack fits

The main complexity is interactive design state and access-controlled project data. A single TypeScript application with a managed Postgres backend addresses both. A separate Express service, Python service, Redis queue, vector database, or AI orchestration framework adds operational work without improving the initial flow.

Vite with Supabase is a viable smaller alternative for a browser-only prototype. Next.js is selected because Florence also needs authenticated print routes, server validation, and a clear place for later share links. Django is viable for a Python-heavy team, but that team preference has not been established. Do not change the chosen stack during a milestone without identifying a concrete blocker.

12 Application architecture

Use server components for authenticated page loading and client components for the palette editor, flower filters, dialogs, and draft state. Keep color and flower scoring functions pure and independent of React or Supabase. No component should contain its own copy of the scoring constants.

Use Route Handlers for project create, save, duplicate, and archive operations. Do not build a parallel set of Server Actions for the same operations. Read operations use a repository module on the server. Pass only serializable data into client components.

```
src/app/(auth)/login/page.tsx
src/app/auth/callback/route.ts
src/app/(workspace)/projects/page.tsx
src/app/(workspace)/projects/new/page.tsx
src/app/(workspace)/projects/[id]/page.tsx
src/app/(workspace)/projects/[id]/print/page.tsx
src/app/(workspace)/flowers/page.tsx
src/app/api/projects/route.ts
src/app/api/projects/[id]/route.ts
src/app/api/projects/[id]/duplicate/route.ts
src/components/ui/
src/features/palette/
src/features/flowers/
src/features/projects/
src/features/concept/
src/lib/color/
src/lib/matching/
src/lib/supabase/
src/lib/validation/
src/lib/repositories/
src/data/demo/
supabase/migrations/
supabase/tests/
tests/unit/
tests/e2e/
```

Feature folders own their components, types, and hooks. Repository modules own database access. Validation modules own Zod schemas. Avoid a generic abstraction framework. Use typed functions with explicit arguments and return values.

Demo mode must be explicit through a local configuration flag. It uses the same schemas and matching functions with an in-memory repository, plus localStorage persistence for demo projects. Show "Demo data saved on this

device.” Never automatically switch a configured production app into demo mode when the backend fails. No auth bypass is allowed in production.

13 Data model

Use relational tables for identity and catalog records. Store the bounded design document in one validated JSONB payload per project. This makes saving a concept atomic and reduces cross-table synchronization for the first release. Do not separately store editable swatches in another table in PO.

Table	Required fields and constraints
workspaces	id UUID primary key; name 1 to 100 chars; currencyCode ISO 4217; timeZone IANA identifier; createdAt
workspace_members	workspaceId and userId composite primary key; role admin, editor, or viewer; userId references auth.users
flower_variants	id text primary key; speciesKey; commonName; variantName; approximateHex; role and style arrays; texture; catalogData JSONB; active boolean; catalogVersion integer
projects	id UUID primary key; workspaceId foreign key; createdBy; name 1 to 100 chars; status draft, ready, or archived; payload JSONB; version integer starting at 1; createdAt; updatedAt

Use snake_case in Postgres and camelCase in TypeScript with an explicit mapping layer. Index projects on workspace_id and updated_at descending. Add a unique index on workspace_members.user_id for the one-workspace-per-user PO contract; remove it only when a workspace switcher is implemented. Catalog size is initially small, so fetch active variants once per session and filter in memory. Revisit search architecture only when measured catalog size or latency requires it.

New projects default to eventType other, environment unknown, style null, eventDate null, empty text fields, no flowers, and Garden Romance with unlocked swatches. Set generationIndex to zero and create the five default element assignments. Duplicate copies get new project and selection IDs and fresh swatch IDs with element references remapped. Truncate the source name as needed so the Copy suffix fits the 100-character limit.

Project payload contract

```
type SwatchRole = 'primary' | 'support' | 'accent' | 'neutral' | 'foliage';
type FlowerRole = 'focal' | 'support' | 'filler' | 'line' | 'foliage';
type Harmony = 'analogous' | 'complementary' | 'splitComplementary'
  | 'triadic' | 'monochromatic';
```

```
interface Swatch {
  id: string;
  role: SwatchRole;
  name: string;
  hex: string;
  locked: boolean;
  proportion: number;
}
```

```
interface SelectedFlower {
  id: string;
  variantId: string;
  role: FlowerRole;
  snapshot: {
    commonName: string;
    variantName: string;
    approximateHex: string;
    imagePath: string | null;
    catalogVersion: number;
  };
  availability: 'unknown' | 'confirmed' | 'unavailable';
  availabilityNote: string;
  availabilityContext: {
    eventDate: string | null;
    location: string | null;
    recordedAt: string;
  };
}
```

```

    } | null;
    practicalChecks: Array<{
      kind: 'scent' | 'exposure' | 'conditioning' | 'supplier';
      status: 'needsReview' | 'reviewed';
      note: string;
    }>;
  }

interface ProjectPayload {
  schemaVersion: 1;
  brief: {
    clientDisplayName: string;
    eventType: 'wedding' | 'corporate' | 'birthday' | 'social' | 'other';
    eventDate: string | null;
    location: string;
    environment: 'indoor' | 'outdoor' | 'mixed' | 'unknown';
    style: 'romantic' | 'garden' | 'modern' | 'classic' | 'vibrant' | null;
    internalNotes: string;
  };
  palette: { mode: Harmony; generationIndex: number; swatches: Swatch[] };
  selectedFlowers: SelectedFlower[];
  elements: Array<{
    key: 'backdrop' | 'linen' | 'floralEmphasis' | 'stationery' | 'accents';
    swatchId: string;
    materialNote: string;
  }>;
  presentationNotes: string;
}

```

Validate exactly five unique swatch roles and IDs, exactly five unique element keys, valid swatch references, normalized HEX, integer proportions totaling 100, unique variant IDs, at most eight selected flowers, and one to four allowed roles per catalog item. Validate chosen roles against the catalog when adding or changing a selected flower.

Limit all notes within SelectedFlower to 500 characters each, swatch names to 40, location and client name to 150, internal notes to 4000, and generationIndex to a nonnegative safe integer. eventDate is a YYYY-MM-DD calendar date, not a UTC timestamp. Validate timestamps and IDs. Limit the entire project update request to 100 KB and reject unknown top-level fields.

Snapshots preserve what the planner selected when catalog content changes. New additions copy a server-verified snapshot from the catalog. Existing snapshots are immutable through normal save requests. Removing an inactive catalog item is allowed; saving an unchanged historical selection is also allowed and displays “Catalog item needs review.” Explicit replacement creates a fresh snapshot.

Ready status means presentation-ready, not supplier-confirmed. Require a valid palette, at least one selected flower, and all five element assignments to mark ready. Archive records the project as archived; restore returns it to draft. No permanent delete control is required in P0.

14 API and persistence contract

All routes authenticate on the server and validate payloads before database work. Never trust workspaceId, createdBy, a role, timestamps, or version increments from the browser. Resolve membership from the authenticated user.

Method and route	Request	Result
POST /api/projects	name and optional brief fields	201 with project and version 1
GET /api/projects	search and status query parameters	200 with current workspace summaries
GET /api/projects/[id]	none	200 with project or 404
PATCH /api/projects/[id]	expectedVersion, name, status, payload	200 with saved project and new version
POST /api/projects/[id]/duplicate	expectedVersion	201 with copied draft

PATCH handles archive and restore through status changes. A viewer receives 403 on mutations. Unauthenticated API requests receive 401; inaccessible project IDs receive 404. Invalid input receives 422 with field errors. A stale version receives 409 with the current version. Backend failure receives a generic 500 response and an internal diagnostic log with no personal notes.

Success envelope: { data: ... }. Error envelope: { error: { code, message, fieldErrors? } }. Use stable codes such as INVALID_INPUT, NOT_AUTHENTICATED, FORBIDDEN, NOT_FOUND, VERSION_CONFLICT, and SAVE_FAILED.

Atomic saves

Provide one Postgres RPC `save_project` that checks the caller's editor or admin membership, verifies the expected version, validates basic database invariants, and updates name, status, payload, version, and `updated_at` in one transaction. Only increment version on a successful mutation. A SQL conditional update must include both project ID and expected version.

Because authenticated users must not bypass server validation by directly updating JSONB, revoke direct INSERT, UPDATE, and DELETE privileges on projects from anon and authenticated. Grant authenticated execution only on carefully scoped create, save, and duplicate RPCs. Those RPCs must enforce the complete payload constraints required for database integrity, not merely trust the Next.js Zod result. Keep validation definitions aligned through shared fixtures and database tests.

If these RPCs use SECURITY DEFINER, set a safe explicit `search_path`, fully qualify objects, derive identity from `auth.uid()`, check membership inside every function, and revoke default PUBLIC execute privileges. RLS remains enabled on readable tables. A function running with elevated rights cannot rely on RLS to perform its own authorization.

Autosave state machine

Valid committed edits become dirty. Debounce 800 ms and send a snapshot with the last confirmed version. Display Unsaved changes, Saving, Saved, or Save failed. Keep partial field edits outside the committed project payload until valid.

Allow only one save request in flight. Edits made during a request remain dirty and are sent after it completes using the new version. Apply the returned version without overwriting newer local draft fields. Do not start another request merely because the response rerendered a component.

For 409, pause autosave and show "This event changed in another session." Offer Reload latest or Save my work as a new project. Never silently overwrite the newer version. For network errors, retain the draft in memory, show Retry, and warn before navigating away. PO does not promise that unsaved production drafts survive closing the browser. On sign-out, clear editor state.

15 Authentication and access control

Use Supabase email sign-in links with public self-registration disabled. Staff accounts and membership records are provisioned by a deployment administrator for the initial release. The product has no invitation-management UI until P1. The first sign-in for an account without membership shows "Your workspace access has not been assigned."

Server routes and print pages must validate authentication, and database access must independently enforce membership. A client-side hidden button is not authorization. Keep authentication refresh logic consistent with the installed Supabase SSR guidance.

Access rules are explicit:

- Members can read only their own workspace row and project rows.
- A user can read their own membership record; ordinary staff cannot list all authentication accounts.
- Admins and editors can create and change projects using authorized RPCs.

- Viewers can read projects and print them.
- Authenticated workspace members can read active catalog entries and historical entries referenced by accessible projects where needed.
- Catalog and membership writes occur through controlled administrative setup, not an app browser session.
- Unauthenticated users have no project or catalog access.

Enable RLS and set appropriate grants together. Policies must be tested for both allowed and denied actions. Avoid recursive membership policies; check membership through a carefully restricted helper or policies that can inspect only the caller’s membership. Supabase documents that grants and row policies serve different purposes, and elevated service credentials bypass RLS. [Supabase row-level security](#)

The browser receives only the project URL and publishable key used by Supabase clients. Never expose a service-role key, database password, or privileged management credential through NEXT_PUBLIC variables. Normal project requests should operate with the user’s session, including calls to the authorized RPCs.

Validate Origin for state-changing browser requests and use the framework’s appropriate cookie protections. Render notes as escaped text; do not render user-provided HTML. Auth redirects must be allowlisted internal destinations. Server-rendered project pages and print output must not enter a shared public cache.

16 Seed content and demonstration scenario

Ship three palette presets with approximate digital colors. These are Florence editorial starting points, not physical flower color standards. Each uses the default five proportions in role order.

Preset	Primary	Support	Accent	Neutral	Foliage
Garden Romance	#D8A7B1	#E8C7B8	#8B4D65	#F4EFE7	#71816A
Modern Evening	#273A53	#A8BAC8	#B78A52	#F3F0E8	#425B4D
Citrus Celebration	#EDB949	#F1AA84	#C85A50	#FFF4DD	#738357

Set the default stored harmony mode for these editorial presets to analogous, but show “Curated preset” immediately after applying one. Do not claim that every preset swatch was generated by that mathematical mode. Selecting Generate alternatives explicitly applies the current mode.

Create the following 16 demo catalog variants. IDs, colors, roles, textures, and tags are fixture data for implementation. Mark editorialReview = pending until a florist checks them. Set all operational facts and stock to unknown. Photographs must come from company-owned or properly licensed files with a credit manifest; otherwise use labeled placeholders.

ID	Variant	HEX	Allowed roles	Texture	Styles
rose-blush	Rose blush	#D8A7B1	focal support	layered	romantic classic
rose-ivory	Rose ivory	#F1E7D8	focal support	layered	romantic classic
rose-burgundy	Rose burgundy	#79384A	focal support	layered	classic modern
carnation-peach	Carnation peach	#E5B39A	support focal	ruffled	romantic garden
carnation-white	Carnation white	#F0EDE4	support	ruffled	classic modern
lisianthus-lilac	Lisianthus lilac	#B6A0C4	support focal	soft	romantic garden
lisianthus-white	Lisianthus white	#F1EEE5	support	soft	classic romantic
dahlia-coral	Dahlia coral	#D97E73	focal	structured	garden vibrant
ranunculus-peach	Ranunculus peach	#E9B59F	focal support	layered	romantic garden

ID	Variant	HEX	Allowed roles	Texture	Styles
gerbera-yellow	Gerbera yellow	#E8BD44	focal support	open	vibrant modern
delphinium-blue	Delphinium blue	#7993BF	line	spired	garden classic
snapdragon-white	Snapdragon white	#EDECE3	line	spired	garden classic
gypsophila-white	Gypsophila white	#EFEFE5	filler	airy	romantic classic
limonium-lavender	Limonium lavender	#A18AB5	filler	airy	garden vibrant
eucalyptus-green	Eucalyptus muted green	#829080	foliage	rounded	garden modern
ruscus-green	Ruscus deep green	#476A48	foliage	pointed	classic modern

Color-family fixture tags follow the variant names: pink for blush, white for ivory and white, red for burgundy and coral, orange for peach, purple for lilac and lavender, yellow, blue, and green as named. Do not derive seasonality or commercial availability from this table.

End to end fixture

Create a demo project named Garden Dinner, with event type social, style romantic, environment indoor, no fixed date, and location Sample venue. Apply Garden Romance. Select rose-blush as focal, lisianthus-white as support, gypsophila-white as filler, and eucalyptus-green as foliage. All practical checks remain Needs review.

Expected behavior: rose-blush receives a ColorFit of 1 against the identical primary HEX; its final Visual fit also depends on role and texture state. The selected flower cards survive save and refresh. The concept uses the default element assignments and prints with four flower records. No availability badge says Confirmed.

17 Error states and quality requirements

Every asynchronous screen needs loading, empty, success, and error states. A failed save cannot be represented by a green Saved indicator. Broken image URLs must fall back to a labeled placeholder. An empty flower search must not reset the project. A session expiring during editing should retain the in-memory draft and offer sign-in again before retrying.

Keyboard users must reach every control. Focus moves into a modal when opened and returns to its trigger when closed. Escape closes non-destructive sheets. Icon-only buttons need accessible names. Swatch selection uses a visible outline and a textual selected state, not color alone. Screen readers hear selected swatch name, HEX, role, and lock state.

Respect reduced-motion preferences. Keep ordinary transitions at 150 to 200 ms. Avoid announcing every picker movement through a live region; announce the committed value and save outcome. Errors must be associated with their fields. Use semantic headings and form labels.

Performance targets are engineering budgets to verify on a representative mobile device: a palette generation should complete below 100 ms for five colors; ranking 500 variants should complete below 150 ms; visible controls should remain responsive while saving. Measure production builds. Do not add complex caching until profiling identifies a bottleneck.

Serve appropriately sized catalog thumbnails and lazy-load below-the-fold imagery. Do not download full-resolution photography for small cards. Log request IDs and failure categories; exclude client names, private notes, session tokens, and full project payloads from diagnostic logs.

18 Acceptance tests

ID	Scenario	Required result
PAL 01	Enter abc as HEX	Stored value becomes #AABBCC
PAL 02	Enter an invalid or eight-digit HEX	Inline error; last valid swatch retained
PAL 03	Lock support and generate	Support HEX remains byte-for-byte identical
PAL 04	Generate twice with identical full inputs	Identical output colors and counter
PAL 05	Generate from white or black	No NaN values; documented neutral fallback
PAL 06	Generate with all colors locked	Action disabled and explanation visible
PAL 07	Enter proportions totaling 117	Save blocked until normalized or corrected
PAL 08	Undo one generation	Previous five swatches and counter restored
FLO 01	Rank identical HEX color	ColorFit equals 1 before other weights
FLO 02	Rank with missing practical data	Unknown status; no invented factual claims
FLO 03	Add the same variant twice	Second addition prevented
FLO 04	Change event date after confirmation	Availability resets to unknown
FLO 05	Remove selected focal flower	Combination advice updates without data loss
FLO 06	Filters return zero records	Empty state and Clear filters shown
SAV 01	Edit during an in-flight save	Later edit persists with next version
SAV 02	Two sessions save the same version	One succeeds and one receives conflict
SAV 03	Network save fails	Draft retained; Retry available; no false Saved
SEC 01	Another workspace requests project ID	No project data returned
SEC 02	Viewer calls mutation directly	Request and RPC both reject it
SEC 03	Authenticated user directly updates table	Direct update denied
SEC 04	Call RPC with forged workspace or malformed payload	Authorization or validation rejection
UI 01	Main flow at 390 and 1440 px	All actions reachable and no body overflow
UI 02	Keyboard-only editing and dialog use	Logical focus and usable controls
EXP 01	Print saved concept	Palette, flowers, assignments, and notes visible
EXP 02	Print with private internal notes present	Internal notes absent from output HTML
EXP 03	Print eight flowers on A4 and Letter	No clipped cards or lost content

Add database tests for anonymous users, members, editors, viewers, and users from another workspace. Use separate browser contexts for conflict and permission tests. Run browser flows in Chromium and WebKit; use Firefox for the main smoke flow where available. Verify the 320 px minimum layout and 200 percent zoom manually in addition to the two main test widths.

19 Implementation milestones for a coding model

Each milestone must leave the app runnable. Do not build P1 features while P0 acceptance criteria are incomplete. After each milestone, report changed files, verification performed, remaining failures, and the next milestone. Do not claim a test passed unless it ran.

Milestone 1 Foundation and fixtures

Create the Next.js TypeScript project, install the chosen dependencies, record versions, create tokens and responsive navigation, define Zod schemas, and add the supplied catalog and project fixtures. Build Projects and the editor shell in explicit demo mode.

Done when the demo opens at 390 and 1440 px, routes work, fixture validation passes, and the application compiles. No backend integration is required yet.

Milestone 2 Palette editor

Implement HEX validation, selected swatch state, color picker, lock controls, five harmony modes, deterministic generation, proportions, presets, and undo. Keep algorithms in pure modules.

Done when PAL 01 through PAL 08 pass and a planner can edit the demo palette using keyboard or touch. Do not proceed with a decorative but nonfunctional palette screen.

Milestone 3 Flower discovery and combinations

Implement catalog cards, detail panels, filtering, ranking, rationale, selected flower roles, practical review fields, and combination advice. Use the same palette state across tabs.

Done when FLO 01 through FLO 06 pass, suggestion reasons correspond to computed features, and there are no unsupported practical claims.

Milestone 4 Concept board and printing

Implement element assignments, presentation notes, the structured preview, and print CSS. Separate internal and external content through an explicit presentation projection function.

Done when EXP 01 through EXP 03 pass with four and eight flowers. Check both color and background-disabled printing.

Milestone 5 Authentication and database

Create migrations, seed records, Supabase SSR authentication, membership setup, policies, validated RPCs, repository implementation, autosave queue, and conflict handling. Preserve the demo repository as a separate local mode.

Done when saving and refresh work, SAV and SEC tests pass, and no production request can bypass membership or payload rules. A service-role key must not be needed for normal editor operations.

Milestone 6 Release verification

Run type checking, linting, unit tests, database authorization tests, browser tests, and a production build. Review accessibility, responsive layouts, error states, print output, and image licensing. Prepare deployment instructions and a backup-and-restore runbook.

Done when every P0 acceptance criterion is satisfied or a specifically documented release blocker remains. Deployment is a separate action; do not publish merely because implementation is finished.

20 Copyable implementation prompt

Use this prompt with the complete Markdown specification attached:

Build Florence according to Florence_Build_Specification.md.
Treat the document as the product and engineering contract.
Read the entire specification before changing files.
Implement P0 only, in the order of the six milestones.
Start with Milestone 1 and stop after its verification report.

Use Next.js App Router, TypeScript, Tailwind, shadcn/ui, Supabase, Zod, and Culori as specified. Use the installed versions consistently and commit the dependency lockfile.
Do not add another backend, state framework, or AI service.

Keep palette and flower algorithms pure and deterministic.
Use the supplied fixtures and label demo content clearly.
Never invent flower stock, prices, seasonality, or safety claims.
Keep all controls functional; omit deferred feature buttons.

Match the responsive layout and tokens in the specification.
Implement loading, empty, error, and permission states.
Do not replace required behavior with hardcoded screenshots.

For each milestone, inspect the existing code, implement only the scoped work, run the relevant checks, and report:

1. What is complete
2. Which tests ran and their results
3. Remaining issues or blockers
4. The exact next milestone

Ask a question only for a material ambiguity that blocks implementation. Otherwise use the documented defaults.
Do not deploy, purchase services, or send invitations.

For each subsequent session, provide the specification plus the last milestone report and say: "Continue with Milestone N. Preserve the existing working behavior. Complete its checks and stop with a verification report."

21 Delivery and handover requirements

The eventual implementation handover must include the repository, setup README, environment variable example without secrets, exact package lockfile, database migrations and seed, documented membership provisioning, tests, catalog image credits, and a concise list of unresolved content checks.

Environment configuration should document NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY, the app origin used for auth redirects, and the local-only demo flag. Privileged migration credentials belong in the deployment operator's environment, not browser configuration or source control.

Use separate development and production Supabase projects. Choose regions near the company's users and consistent with its data requirements. Verify the selected database plan's backup and restore capability, document retention, and test restoring a non-production copy before storing important live projects. Keep schema migrations versioned and test them against an empty database.

Before production use, the company should supply its name and brand assets, approved flower imagery, initial staff access list, preferred currency and time zone, and florist-reviewed catalog corrections. These inputs do not block the demo implementation. They determine the production content and operating setup.

The product is ready for a pilot when a planner can complete the main flow, saved choices survive refresh, access isolation is proven, recommendations explain their visual basis, unresolved flower facts remain visible, and the exported concept is readable on paper.